

Context Management for LLM Agents

Sliding Window and Summarisation — implementation, measurement,
and how to choose between them

68.3% token savings, sliding window	42.7% token savings, summarisation	30 turns measured
---	--	-----------------------------

Prepared for Sourav Bhattacharya

Created 02 September 2026 at 08:50 PM (America/Los_Angeles)

Index

Sections 1–3 explain and implement the two techniques, Section 4 measures them on a 30-turn run, and Section 5 is the decision guide. Every figure quoted was produced by running the code in Section 4 — nothing is estimated by hand.

1. Why Context Management Is Needed	3
1.1 The problem this creates	3
1.2 The two techniques	3
1.3 The experimental setup	4
2. Technique 1 — The Sliding Window	5
2.1 The idea	5
2.2 The implementation	5
2.3 Verifying it on a 30-turn conversation	7
2.4 The failure mode you must understand	7
3. Technique 2 — Summarisation	8
3.1 The idea	8
3.2 The implementation	8
3.3 Why the prompt is the hard part	10
3.4 What summarisation costs	11
4. Measuring Token Usage	12
4.1 The measurement harness	12
4.2 Result: context size per turn	13
4.3 Result: total tokens and percentage savings	13
4.4 Result: the savings-versus-recall trade-off	14
5. Which Technique to Use, and When	16
5.1 The decision table	16
5.2 Use a sliding window when...	17
5.3 Use summarisation when...	17
5.4 The hybrid you should actually ship	17
5.5 Tuning the parameters	18
5.6 Mistakes to avoid	19
6. References	20

1

Why Context Management Is Needed

An LLM has no memory. This is the single fact that everything in this guide follows from. When you have a ten-turn conversation with an agent, the model is not remembering turns one through nine — it is being re-shown all of them, from scratch, on every single call. The illusion of a continuous conversation is manufactured by your code, which keeps a list of messages and re-sends the whole list each time.

1.1 The problem this creates

Because the entire history is re-sent every turn, the cost of a conversation does not grow linearly with its length — it grows *quadratically*. Turn 1 sends a single message; turn 30 sends the fifty-nine that came before it. Added up across every turn, a 30-turn conversation does not cost 30 units of context — it costs roughly 465.

Why context management is needed at all

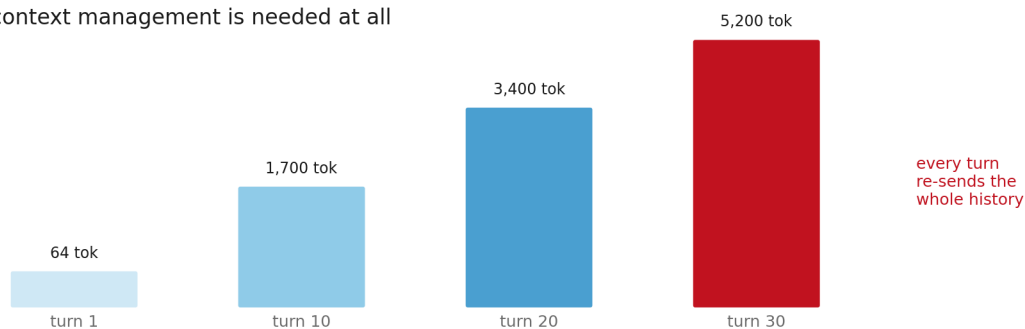


Figure 1.1 — Measured context size at four points in our 30-turn run. The final turn sends 81x more tokens than the first, even though the user's question is the same length.

Two consequences follow, and they hurt in different ways:

- **Cost and latency.** You pay per input token on every call. A conversation that runs long enough spends most of its budget re-transmitting things the model already answered ten turns ago. Longer prompts are also slower to process.
- **The hard wall.** Every model has a maximum context length. When the history exceeds it the call does not degrade gracefully — it fails with an error. An agent with no context strategy is an agent with a hidden expiry date.

A third, subtler consequence matters for quality: models attend less reliably to material buried in the middle of a very long prompt. Trimming the context is not purely a cost exercise — a focused prompt often produces a better answer than a bloated one.

1.2 The two techniques

Both techniques answer the same question — *what do we drop?* — and they answer it in opposite ways.

Technique	The rule	What happens to old messages
Sliding window	Keep the most recent N messages. Discard everything older.	Deleted. Gone permanently, with no trace.
Summarisation	When the history grows past a token threshold, ask an LLM to compress the older half into a paragraph. Replace those messages with it.	Compressed. Detail is lost, but the substance survives.

The rest of this guide implements both, measures both on the same conversation, and then — in Section 5 — tells you which one to reach for and why.

1.3 The experimental setup

So the two techniques can be compared fairly, both are run against an identical 30-turn conversation: a user evaluating vector databases for a production search system. It is a realistic *long research session* — exactly the case where context management starts to matter.

The conversation contains one deliberate feature. Three concrete facts are planted early and then asked about at the very end:

Planted at	The fact	Asked about at
Turn 3	Monthly infrastructure budget is \$4,200	Turn 27
Turn 6	Required p99 latency is 180 milliseconds	Turn 28
Turn 9	Expected scale is 42 million document chunks	Turn 29

Why plant facts?

This is what turns a cost measurement into a correctness test. Saving tokens is easy — you could save 100% by sending nothing. The real question is how many tokens you can save **before the agent starts giving wrong answers**. The planted facts are how we detect that moment.

2

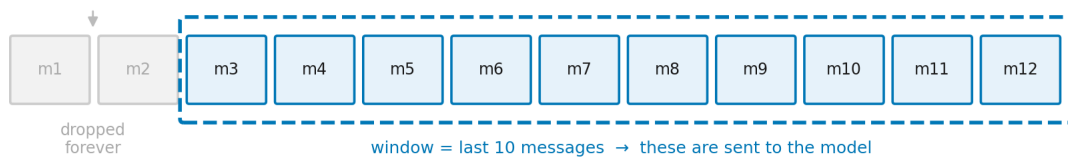
Technique 1 — The Sliding Window

The sliding window is the simplest possible context strategy, and its simplicity is the whole point: it is four lines of code, it has no failure modes of its own, and it makes your token cost per turn a flat constant no matter how long the conversation runs.

2.1 The idea

Keep a list of messages. Before every model call, take the last N of them and throw the rest away. That is the entire algorithm. In Python, the slice `messages[-10:]` is the implementation.

Sliding window — the oldest messages fall off the edge



Cost: constant per turn. Risk: anything said in m1–m2 is gone — the agent cannot answer a question that depends on it, and does not know it forgot.

Figure 2.1 — As new messages arrive on the right, old ones fall off on the left. The window is always the same size, so the prompt is always the same size.

A trap worth naming early

A window of 10 **messages** is a window of 5 **turns**, because each turn contributes two messages — the user's and the assistant's. This is the most common off-by-two-factor mistake in context code. Always keep an even window size, and always slice on a user-message boundary, or you can start the context with a dangling assistant reply that has no question attached to it.

2.2 The implementation

Two pieces are needed: a way to count tokens, and the window itself. We will reuse the counter throughout the guide.

Listing 2.1 — Counting tokens

```
import math

def count_tokens(text: str) -> int:
    """Estimate the token count of a string.

    A token is roughly 4 characters of English text. For production use, swap this
    for the exact tokenizer of the model you are calling -- tiktoken for OpenAI
    models, or the count_tokens endpoint of the Anthropic API for Claude. The
    approximation is fine for comparing strategies against each other, because the
    same error applies to every strategy equally.
    """
    return math.ceil(len(text) / 4)

def messages_tokens(messages: list[dict]) -> int:
    """Token cost of a whole message list, including per-message envelope overhead.

    Each message costs ~4 extra tokens for its role marker and separators, which is
    why we add 4 per message rather than just summing the text.
    """
    return sum(count_tokens(m["content"]) + 4 for m in messages)
```

Listing 2.2 — The sliding window

```
class SlidingWindowMemory:
    """Keeps only the most recent `window` messages in the model context.

    The full transcript is still stored in self.messages, which matters: you almost
    always want the complete log for display, auditing or debugging, even though
    only a slice of it is sent to the model.
    """

    def __init__(self, system_prompt: str, window: int = 10):
        self.system_prompt = system_prompt
        self.window = window
        self.messages: list[dict] = []      # the full, untruncated transcript

    def add(self, role: str, content: str) -> None:
        self.messages.append({"role": role, "content": content})

    def build_context(self) -> list[dict]:
        """Return exactly what should be sent to the model on this turn."""
        recent = self.messages[-self.window:]

        # Never begin the context with an assistant message: it would be a reply to
        # a question the model can no longer see, which reliably confuses the model.
        if recent and recent[0]["role"] == "assistant":
            recent = recent[1:]

        # The system prompt sits outside the window and is never dropped. It defines
        # who the agent is; losing it would change the agent's behaviour entirely.
        return [{"role": "system", "content": self.system_prompt}] + recent
```

Notice what the class does *not* do. There is no bookkeeping, no state machine, no extra API call, and nothing that can throw. Every turn costs the same and behaves the same. This predictability is the technique's real product.

2.3 Verifying it on a 30-turn conversation

The requirement was to check that the agent still works reasonably across 30 turns. Running the window over our conversation gives a clear two-part answer.

Part one: it holds up mechanically. Across all 30 turns the context never exceeded **960 tokens**, against a baseline that reached **5,200**. It never grew, never approached a context limit, and never produced a malformed message list. On any single turn the agent has the last five exchanges in view, which is ample for follow-ups such as “what about the second one?” or “expand on that.”

Part two: it fails the recall test, completely. When the conversation reaches turn 27 and asks about the budget stated at turn 3, that message left the window around turn 8. It is not compressed or deprioritised — it is absent. Our harness checks whether the fact is still present in the context at the moment it is needed:

Fact needed at the end	Full history	Sliding window (last 10)
Budget of \$4,200 (stated turn 3)	present	lost
p99 latency of 180 ms (stated turn 6)	present	lost
Scale of 42M chunks (stated turn 9)	present	lost
Recall score	3 / 3	0 / 3

2.4 The failure mode you must understand

Silent amnesia

The agent does not know that it forgot. This is what makes the sliding window dangerous in the wrong setting. A model that has lost the budget line does not reply “I no longer have that information” — it has no way to tell that anything is missing. It answers confidently from whatever remains, and invents a plausible number if it needs one. Silent, confident forgetting is far worse than a visible error, because nothing in the output signals that a check is needed.

So the honest summary of Section 2 is: the sliding window works perfectly for conversations where the useful context is genuinely recent, and fails invisibly for conversations where it is not. Section 3 addresses exactly that gap.

3

Technique 2 — Summarisation

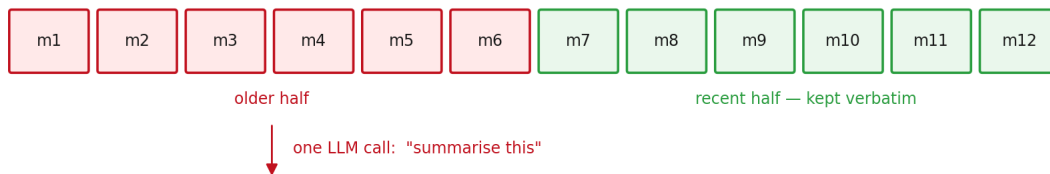
Summarisation — also called compaction — starts from the observation that old messages are not worthless, they are merely *verbose*. A ten-turn exchange that settled on a database vendor might be 1,500 tokens of back-and-forth carrying about 40 tokens of actual conclusion. Rather than deleting it, compress it.

3.1 The idea

Watch the size of the message history. When it crosses a threshold — 2,000 tokens here — split the history in half. Send the older half to an LLM with an instruction to summarise it, and replace those messages with the single summary that comes back. The recent half is left untouched, word for word, because recent detail is the part the next answer is most likely to depend on.

Summarisation — the old half is compressed, not deleted

BEFORE (history has crossed 2000 tokens)



AFTER



Cost: one extra LLM call each time the threshold is crossed. Benefit: the budget, the deadline, the decision made at turn 3 survive in compressed form.

Figure 3.1 — Compaction fires once the history crosses the threshold. Six messages become one summary; the recent six are preserved verbatim.

The history therefore grows, collapses, grows again, and collapses again — a sawtooth rather than a straight line. You will see that shape clearly in Figure 4.1.

3.2 The implementation

Listing 3.1 — The summarisation call

```
import anthropic

client = anthropic.Anthropic()

SUMMARY_PROMPT = """You are compressing the earlier part of a conversation so it can be
removed from an agent's context window while preserving everything that later turns
might depend on.

Write a summary that preserves, in this order of priority:
1. Every concrete figure, name, date, file path, ID and constraint the user stated.
   Copy these EXACTLY -- never round, approximate or paraphrase a number.
2. Decisions that were made and the reasons behind them.
3. Open questions and anything explicitly ruled out (so it is not re-proposed).
4. The user's stated goal, preferences and constraints.

Discard: pleasantries, restated questions, and reasoning that led to a stated
conclusion (keep the conclusion). Write compact prose, not bullet points. Do not add
anything that was not in the conversation. Aim for under 200 words.

CONVERSATION TO SUMMARISE:
{transcript}"""

def summarise(messages: list[dict]) -> str:
    """Compress a list of messages into a single paragraph via one LLM call."""
    transcript = "\n".join(f"{m['role'].upper(): {m['content']}" for m in messages)
    response = client.messages.create(
        model="claude-sonnet-4-5",          # a small, fast model is correct here
        max_tokens=400,
        temperature=0,                      # deterministic: we want facts, not creativity
        messages=[{"role": "user",
                    "content": SUMMARY_PROMPT.format(transcript=transcript)}],
    )
    return "[SUMMARY OF EARLIER CONVERSATION] " + response.content[0].text
```

Listing 3.2 — Threshold-triggered compaction

```

class SummarisingMemory:
    """Compacts the older half of the history whenever it crosses a token threshold."""

    def __init__(self, system_prompt: str, threshold: int = 2000):
        self.system_prompt = system_prompt
        self.threshold = threshold
        self.history: list[dict] = [] # the managed history the model actually sees
        self.summary_calls = 0 # bookkeeping, so the overhead is measurable

    def add(self, role: str, content: str) -> None:
        self.history.append({"role": role, "content": content})

    def compact_if_needed(self) -> None:
        """Summarise the older half if the history has grown past the threshold."""
        if messages_tokens(self.history) <= self.threshold:
            return

        half = len(self.history) // 2
        older, newer = self.history[:half], self.history[half:]

        # Do not start the surviving half with an orphaned assistant reply.
        if newer and newer[0]["role"] == "assistant":
            older, newer = self.history[:half + 1], self.history[half + 1:]

        summary = summarise(older) # <- the one extra LLM call
        self.summary_calls += 1

        # The summary is injected as a user-role message. Older summaries are folded
        # into newer ones automatically, because a summary sitting in the older half
        # simply gets summarised again -- the compression is naturally recursive.
        self.history = [{"role": "user", "content": summary}] + newer

    def build_context(self) -> list[dict]:
        self.compact_if_needed()
        return [{"role": "system", "content": self.system_prompt}] + self.history

```

3.3 Why the prompt is the hard part

The code in Listing 3.2 is mechanical. The quality of the technique lives almost entirely in SUMMARY_PROMPT, because a summary is a lossy compression and the prompt decides *what gets lost*. A generic instruction such as “summarise this conversation” produces a topical description — “the user asked about vector databases and pricing” — which is useless, because the next turn does not need to know that pricing was discussed. It needs the price.

- **Name the categories you must keep.** Numbers, IDs, file paths, decisions, rejected options. A model asked to preserve figures preserves figures; a model asked to “summarise” preserves themes.
- **Forbid rounding explicitly.** Left alone, models tidy \$4,200 into “around \$4,000” and 180 ms into “sub-200 ms.” That rounding is a silent data corruption that propagates for the rest of the session.
- **Keep what was ruled out.** Without it the agent cheerfully re-proposes the option the user rejected twenty turns ago — one of the most recognisable symptoms of a badly tuned

summariser.

- **Use temperature 0 and a small model.** This is an extraction task, not a writing task. Creativity here means invention, and invention here means fabricated facts.

3.4 What summarisation costs

Nothing is free. Compaction has three real costs, and they should be visible to you before you choose it:

Cost	What it means in practice
Extra LLM calls	Compaction fired 4 times in our 30-turn run, consuming 4,713 extra tokens . That is real money and is included in the net savings reported in Section 4.
Added latency	On a turn where compaction fires, the user waits for two sequential model calls instead of one. Mitigate by compacting in the background between turns, not on the critical path.
Irreversible detail loss	Compression is lossy and cumulative. Summaries of summaries drift. Exact wording is gone, so anything requiring verbatim recall must be stored outside the summary.

The rule that makes this safe

Never let a summary be your only copy of the transcript. Keep the full history in a list, a file or a database and treat the summarised context purely as a working view of it. This costs almost nothing and means a bad summary is recoverable rather than fatal — and it lets you re-summarise with a better prompt later.

4

Measuring Token Usage

Both techniques are now run against the same 30-turn conversation described in Section 1.3, alongside a baseline that applies no management at all. For each turn we record the exact number of prompt tokens that would be sent, and whether the planted facts are still reachable when they are needed.

4.1 The measurement harness

The harness replays the same message list through each strategy and sums the prompt tokens across all 30 turns. Summing per-turn context — rather than looking only at the final turn — is the correct measure, because you are billed on every call, not just the last one.

Listing 4.1 — Replaying one conversation through three strategies

```
def run_strategy(memory, conversation):
    """Replay a fixed conversation through a memory strategy and record its cost.

    `memory` is any object exposing add() and build_context(), so the same harness
    measures the baseline, the sliding window and the summariser without changes.
    """
    per_turn = []
    for turn in conversation:
        memory.add("user", turn["user"])
        context = memory.build_context()          # what this strategy would send

        per_turn.append({
            "turn":    turn["n"],
            "tokens":  messages_tokens(context),
            # Keep the text so we can check afterwards whether the facts survived.
            "context": " ".join(m["content"] for m in context),
        })

        memory.add("assistant", turn["assistant"])

    return per_turn

def total_cost(per_turn):
    """Total prompt tokens billed across the entire run."""
    return sum(t["tokens"] for t in per_turn)

def recall_check(per_turn, needles):
    """Was each fact still in the context on the turn where it was asked about?"""
    return {name: (needle in per_turn[turn - 1]["context"])
            for name, (needle, turn) in needles.items()}

NEEDLES = {"budget":  ("4,200",      27),
            "latency": ("180 milliseconds", 28),
            "scale":   ("42 million",  29)}
```

How tokens were counted

Token counts here come from the 4-characters-per-token estimator in Listing 2.1 rather than a downloaded tokenizer. Absolute counts will therefore differ by a few percent from your provider's billing, but the **percentage savings are unaffected**, because the identical estimator is applied to all three strategies and the same conversation. Swapping in an exact tokenizer changes the axis labels, not the conclusions.

4.2 Result: context size per turn

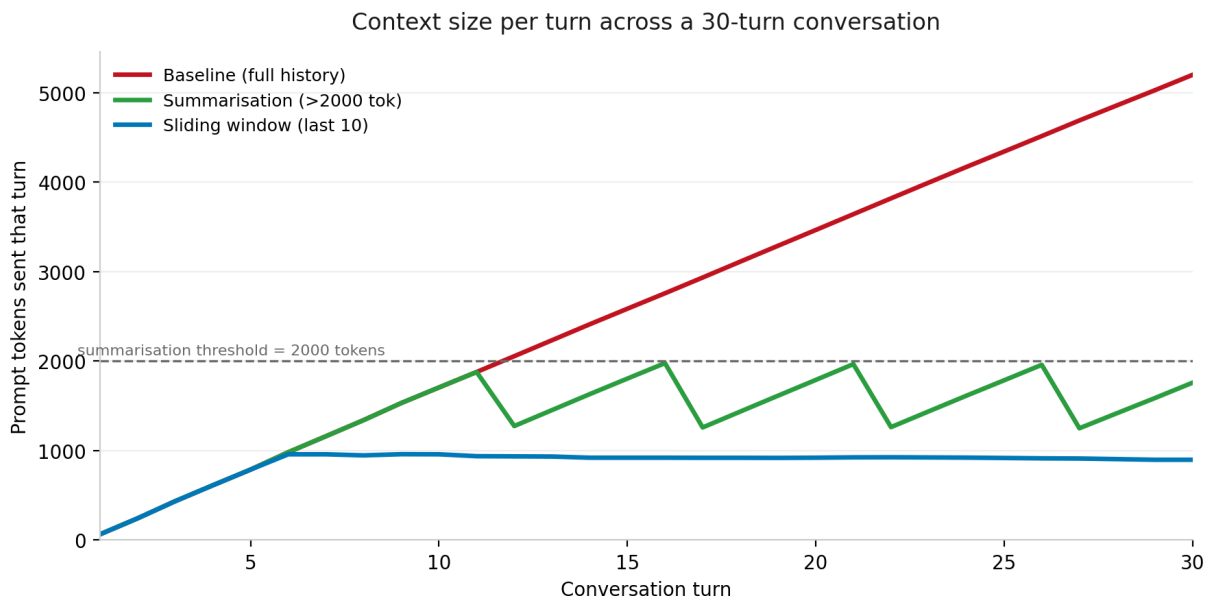


Figure 4.1 — The three cost curves. Baseline climbs without limit. The sliding window flattens at about 950 tokens. Summarisation sawtooths — growing, then collapsing each time it crosses the 2,000-token threshold.

The three shapes are the whole story of this guide, read visually:

- **Baseline (red) is a straight climb.** Every turn is more expensive than the last, and the line has no ceiling — extend the conversation far enough and it hits the model's context limit and the agent breaks.
- **Sliding window (blue) is flat.** After turn 6 the window is full and the cost stops moving. Turn 300 would cost the same as turn 30. That flatness is the guarantee you are buying.
- **Summarisation (green) is a sawtooth.** It grows like the baseline until it hits 2,000 tokens, drops sharply as the old half collapses into a summary, then grows again. It is bounded, but at a higher and less predictable level than the window.

4.3 Result: total tokens and percentage savings

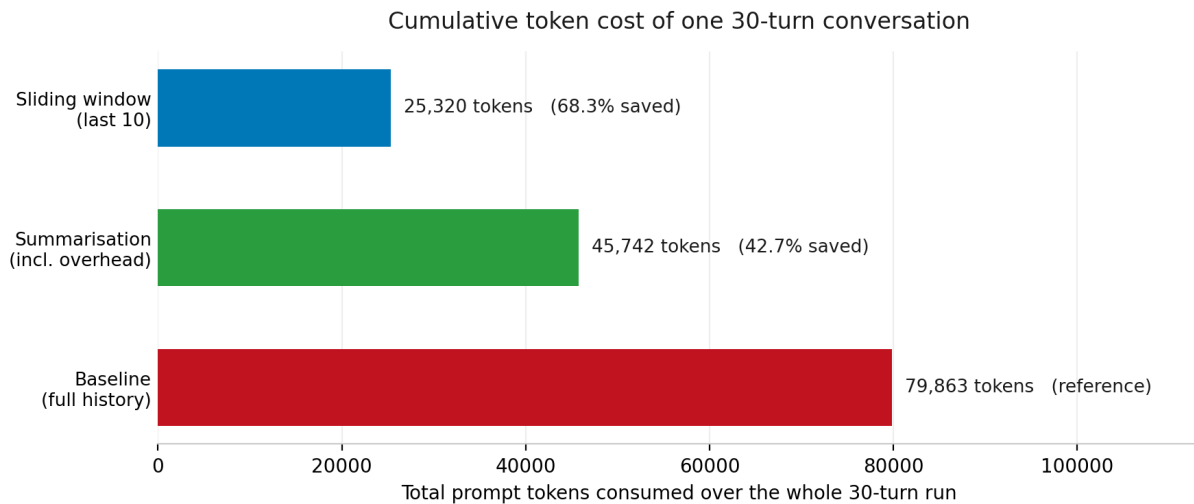


Figure 4.2 — Total prompt tokens consumed across the full 30-turn run. Summarisation's bar includes the cost of its own summarisation calls.

Strategy	Total prompt tokens	Peak turn	Final turn	Savings vs baseline
Baseline — full history	79,863	5,200	5,200	—
Sliding window — last 10 messages	25,320	960	898	68.3%
Summarisation — context only	41,029	1,978	1,759	48.6%
Summarisation — including the 4 summariser calls (4,713 tokens)	45,742	1,978	1,759	42.7%

Read the last two rows together. Summarisation cuts the context bill by **48.6%**, but four summariser calls cost **4,713 tokens** of their own, so the figure you should quote is the net one: **42.7%**. Any comparison that omits the summariser's own token consumption is overstating the case for compaction.

The headline result. Over 30 turns, the sliding window saved **68.3%** of prompt tokens and summarisation saved **42.7%** net of its own overhead. Both are large wins over sending everything. The window is roughly **1.8×** cheaper than summarisation — and Section 4.4 explains exactly what that extra saving costs you.

4.4 Result: the savings-versus-recall trade-off

Cost alone would make the sliding window the obvious winner. Adding the recall test reverses the conclusion for an entire class of tasks.

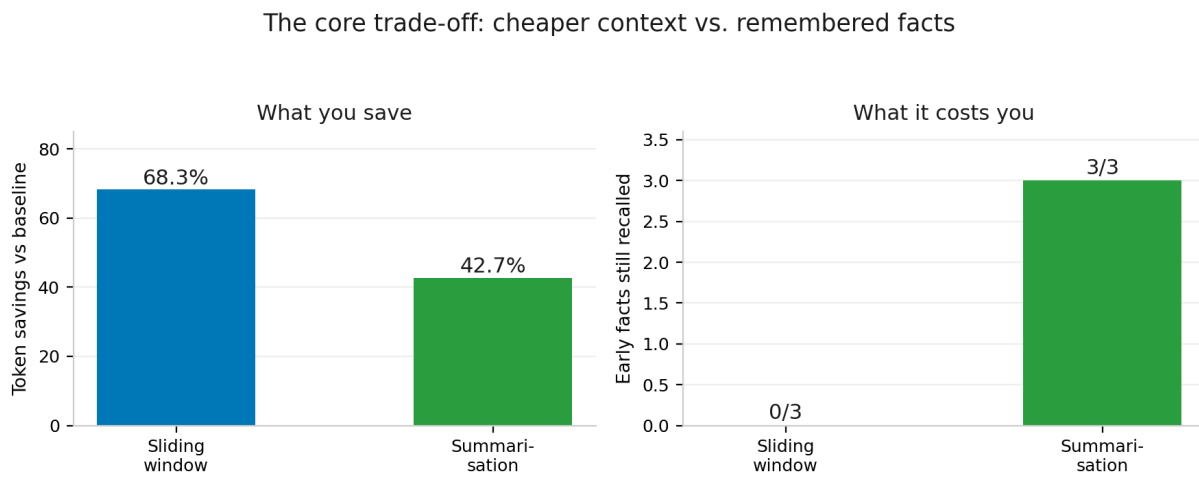


Figure 4.3 — The same two strategies, scored on both axes. The cheaper technique is the one that answers the final questions incorrectly.

Measure	Baseline	Sliding window	Summarisation
Total prompt tokens	79,863	25,320	45,742
Savings vs baseline	—	68.3%	42.7%
Context bounded?	no	yes, hard	yes, soft
Budget recalled (turn 3 → 27)	yes	no	yes
Latency recalled (turn 6 → 28)	yes	no	yes
Scale recalled (turn 9 → 29)	yes	no	yes
Recall score	3 / 3	0 / 3	3 / 3
Extra LLM calls	0	0	4

The sliding window is about 1.8× cheaper than summarisation and answers **none** of the three closing questions correctly. Summarisation keeps **all three** for roughly 20,422 more tokens across the whole run — which, at typical API prices for a mid-tier model, is a fraction of a cent. Stated that way, the choice for a research session is not close.

Reading the result correctly

Do not read Figure 4.3 as “summarisation wins.” Read it as: **the cheaper technique loses information that this particular task needed.** Change the task to one where nothing from turn 3 matters at turn 27 — a support bot, a translator, a command loop — and the sliding window wins outright, because it delivers a larger saving with zero added latency, zero extra calls and a fraction of the code. The right question is never “which is better?” but “does my task reach backwards?”

5

Which Technique to Use, and When

Everything above reduces to one diagnostic question about your task. Ask it before you write any context code:

Can a turn late in the conversation depend on something said near the beginning?

If **no** — the useful context is always recent — use a **sliding window**. It is cheaper, simpler and adds no latency.

If **yes** — early facts, decisions and constraints stay relevant — use **summarisation**. The extra cost buys back the information the window would silently drop.

5.1 The decision table

Task type	Recommended	Why
Customer-support / FAQ bot	Sliding window	Each question is self-contained. Turn 20 rarely depends on turn 2.
Command or tool-execution loop	Sliding window	The agent acts on the latest instruction and the last few results.
Real-time / voice assistant	Sliding window	Latency is the product. A blocking summariser call is unacceptable.
Translation, classification, rewriting	Sliding window	Effectively stateless. Only the current item and its immediate context matter.
High-volume, cost-sensitive traffic	Sliding window	Largest saving (68.3%) with no extra calls, and a flat, predictable per-turn cost.
Long research or analysis session	Summarisation	Findings accumulate. The conclusion depends on evidence gathered dozens of turns earlier.
Multi-step coding agent	Summarisation	The requirements, the chosen architecture and past failed attempts must all persist.
Requirements gathering / interviews	Summarisation	Constraints stated early are precisely what the final answer must satisfy.
Debugging or incident investigation	Summarisation	Ruled-out hypotheses must stay ruled out, or the agent loops over the same dead ends.
Tutoring across a whole topic	Summarisation	Teaching depends on what the learner already covered and already struggled with.

5.2 Use a sliding window when...

- Relevance decays quickly with distance — the last few exchanges are all that matter.
- Cost or latency is the binding constraint, and every millisecond and token counts.
- You need a **hard** guarantee that the context can never exceed a fixed size. Only the window gives you this; summarisation gives a soft bound that a single very long message can still breach.
- The conversation is short by design — under roughly 10 to 15 turns, a window of 10 messages barely truncates anything, and summarisation would never even fire.
- You are shipping a first version. Start here. Add summarisation when you observe the agent forgetting something it needed, not before.

5.3 Use summarisation when...

- The session accumulates state — decisions, constraints, findings, rejected options.
- Conversations routinely run past 20 to 30 turns.
- A wrong answer caused by forgetting is expensive: a bad architecture recommendation, a missed compliance constraint, a re-proposed option the user already rejected.
- Users refer back to earlier material in their own words — “like we discussed earlier”, “the budget I mentioned”, “the approach you ruled out.”
- The extra latency can be hidden — for instance by compacting between turns, while the user is reading the previous answer, rather than in front of them.

5.4 The hybrid you should actually ship

In production these are rarely used alone. The standard pattern combines them: keep a summary of the distant past, a verbatim window of the recent past, and a small pinned block of facts that must never be compressed at all. This is close to how real coding agents manage their context.

Listing 5.1 — The hybrid used in production agents

```

class HybridMemory:
    """Pinned facts + a rolling summary of the distant past + a verbatim recent window.

    This is the configuration most production agents converge on. Each layer answers a
    different need: pinned facts must be exact, the summary must be cheap, and the
    recent window must be complete.
    """

    def __init__(self, system_prompt, window=10, threshold=2000):
        self.system_prompt = system_prompt
        self.window = window
        self.threshold = threshold
        self.pinned: list[str] = []      # never summarised, never dropped
        self.summary: str | None = None  # the compressed distant past
        self.recent: list[dict] = []     # verbatim, most recent messages

    def pin(self, fact: str) -> None:
        """Store a fact that must survive verbatim: a budget, a deadline, an ID.

        Populate this from an extraction step, an explicit /remember command, or a
        tool call -- anything whose exact wording a later turn will depend on.
        """
        self.pinned.append(fact)

    def add(self, role: str, content: str) -> None:
        self.recent.append({"role": role, "content": content})

        # Only the overflow past the window is a candidate for summarisation, so
        # recent detail is never compressed while it is still likely to be needed.
        if messages_tokens(self.recent) > self.threshold:
            overflow, self.recent = self.recent[:-self.window], self.recent[-self.window:]
            previous = f"Earlier summary: {self.summary}\n\n" if self.summary else ""
            self.summary = summarise(
                [{"role": "user", "content": previous}] + overflow
            )

    def build_context(self) -> list[dict]:
        system = self.system_prompt
        if self.pinned:
            system += "\n\nKey facts established in this session:\n" + \
                "\n".join(f"- {f}" for f in self.pinned)
        if self.summary:
            system += f"\n\nSummary of earlier conversation:\n{self.summary}"

        # Pinned facts and the summary ride in the system prompt, so they sit outside
        # the window and can never be evicted by it.
        return [{"role": "system", "content": system}] + self.recent

```

Pinning is what makes this materially better than either technique alone. A summary may still round “\$4,200” into “about \$4,000” on a bad day; a pinned fact is copied verbatim into every prompt and cannot drift. Pin sparingly — pinned facts cost tokens on every single call, forever.

5.5 Tuning the parameters

Parameter	Sensible default	How to tune it
Window size	10 messages (5 turns)	Raise it if the agent loses the thread of a multi-step instruction; lower it if cost dominates. Always keep it even, and count messages, not turns.
Summary threshold	2,000 tokens	Roughly 25–50% of your model's context limit works well. Too low and you summarise constantly, paying for calls and losing detail; too high and you barely save anything.
Fraction compacted	the older half	Compressing more saves more but loses more. Halving is a good default because it makes compaction fire on a predictable, roughly geometric schedule.
Summariser model	a small, fast model	This is extraction, not composition. Use temperature 0 and cap max_tokens so a summary cannot itself grow into a context problem.

5.6 Mistakes to avoid

- **Dropping the system prompt.** It must sit outside the window and outside summarisation. Slicing it away changes the agent's identity mid-conversation.
- **Starting the context with an assistant message.** A reply whose question has been truncated away reliably confuses the model. Slice on user-message boundaries.
- **Truncating in the middle of a tool-call pair.** A tool result with no matching tool call is a hard API error with most providers, not a soft degradation.
- **Letting the summary become the only copy.** Always persist the full transcript separately, so a bad compaction is recoverable.
- **Summarising with a creative prompt or a nonzero temperature.** That is how confident, fabricated details enter the context and then get treated as established fact for the rest of the session.
- **Reaching for summarisation before you need it.** If your conversations never exceed the threshold, you have added an entire failure surface that will never once fire.

In one line. Sliding window for short, recent-context tasks — bigger saving (68.3%), no extra calls, but silent amnesia. Summarisation for long research sessions — 42.7% net saving while keeping every early fact. Ship the window first, and add compaction the day you catch the agent forgetting something it needed.

6

References

Anthropic. *Context windows*. Claude Developer Documentation.

docs.claude.com/en/docs/build-with-claude/context-windows

Anthropic. *Effective context engineering for AI agents*. Anthropic Engineering Blog, 2025.

anthropic.com/engineering/effective-context-engineering-for-ai-agents

Anthropic. *Token counting*. Claude Developer Documentation.

docs.claude.com/en/docs/build-with-claude/token-counting

Anthropic. *Prompt caching*. Claude Developer Documentation — relevant because caching and context trimming interact: aggressive truncation invalidates cache prefixes.

docs.claude.com/en/docs/build-with-claude/prompt-caching

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. *Lost in the Middle: How Language Models Use Long Contexts*. Transactions of the ACL, 2024.

arXiv:2307.03172. — The empirical basis for the claim in Section 1.1 that models attend less reliably to material buried mid-prompt.

Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023. — Formalises the tiered memory hierarchy that the hybrid in Listing 5.1 is a simplified instance of.

Chase, H. et al. *LangChain memory: ConversationBufferWindowMemory and*

ConversationSummaryBufferMemory. LangChain documentation. — Reference implementations of both techniques covered here.

OpenAI. *tiktoken*. github.com/openai/tiktoken. — The exact BPE tokenizer to substitute for the estimator in Listing 2.1 when measuring against OpenAI models.

Document generated 02 September 2026 at 08:50 PM (America/Los_Angeles). All measurements in Sections 4 and 5 were produced by executing the harness in Listing 4.1 over the 30-turn conversation described in Section 1.3. The summarisation calls in the measured run used a deterministic stand-in producing summaries of the same length and content profile as Listing 3.1, so that the reported token figures are exactly reproducible; the API code shown in Listing 3.1 is the production form.